



Rust

Cristiano Damiani Vasconcellos
cristiano.vasconcellos@udesc.br

struct (Produto)

```
#[derive(Debug, Clone, Copy)]
struct Ponto {
    x: u32, y: u32
}

fn main() {
    let mut p1 = Ponto {x:1, y:10};
    p1.x = 200;
    println!("{p1:?}");
    let p2 = p1;
    println!("{p2:?}");
    println!("{p1:?}");
    let p3 = &p1;
    println!("{}", p3.y);
    p3.y = 12; //Erro
    println!("{p3:?}");
}
```

enum (União Disjunta)

```
#[derive(Debug)]
enum Semana {
    Dom, Seg, Ter, Qua, Qui, Sex, Sab
}

use Semana::{*};

fn prox_dia_util(d:Semana) -> Semana {
    match d {
        Seg => Ter,
        Ter => Qua,
        Qua => Qui,
        Qui => Sex,
        _   => Seg
    }
}

fn main (){
    let hoje = Semana::Qui;
    println!("{hoje:?}");
    println!("{:?}", prox_dia_util(hoje));
}
```

enum (União Disjunta)

```
#[derive(Debug)]
enum Semana {
    Dom, Seg, Ter, Qua, Qui, Sex, Sab
}

use Semana::{*};

fn prox_dia_util(d:Semana) -> Semana {
    match d {
        Seg => return Ter,
        Ter => return Qua,
        Qua => return Qui,
        Qui => return Sex,
        _   => return Seg
    }
}

fn main (){
    let hoje = Semana::Qui;
    println!("{hoje:?}");
    println!("{:?}", prox_dia_util(hoje));
}
```

enum com dados (União Disjunta)

```
use std::f64::consts::PI;
use Forma::{*};

#[derive(Debug, Clone, Copy)]
enum Forma {
    Circulo {raio:f64},
    Retangulo {alt:f64, compr:f64}
}

fn area(f:Forma) -> f64 {
    match f {
        Circulo {raio: r} => r * r * PI,
        Retangulo {alt: a, compr: c} => a * c
    }
}

fn main (){
    let c = Circulo {raio:2f64};
    println!("{c:?}");
    println!("{}", area(c));
    println!("{c:?}");
}
```

enum com dados (União Disjunta)

```
use std::f64::consts::PI;
use Forma::*;

#[derive(Debug)]
enum Forma {
    Circulo {raio:f64},
    Retangulo {alt:f64, compr:f64}
}

fn area(f:&Forma) -> f64 {
    match *f {
        Circulo {raio: r} => r * r * PI,
        Retangulo {alt: a, compr: c} => a * c
    }
}

fn main (){
    let c = Circulo {raio:2f64};
    println!("{c:?}");
    println!("{}", area(&c));
    println!("{c:?}");
}
```

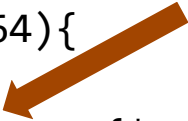
Funções Associadas

```
use std::f64::consts::PI;
use Forma::*;
#[derive(Debug)]
enum Forma {
    Circulo {raio:f64},
    Retangulo {alt:f64, compr:f64}
}
impl Forma {
    fn area(f:&Forma) -> f64 {
        match *f {
            Circulo {raio: r} => r * r * PI,
            Retangulo {alt: a, compr: c} => a * c
        }
    }
}
fn main (){
    let r = Retangulo {alt:10f64, compr:5f64};
    println!("{r:?}");
    println!("{}", Forma::area(&r));
    println!("{r:?}");
}
```

```
use std::f64::consts::PI;
use Forma::*;
#[derive(Debug)]
enum Forma {
    Circulo {raio:f64},
    Retangulo {alt:f64, compr:f64}
}
impl Forma {
    fn area(self) -> f64 {
        match *self {
            Circulo {raio: r} => r * r * PI,
            Retangulo {alt: a, compr: c} => a * c
        }
    }
}
fn main (){
    let r = Retangulo {alt:10f64, compr:5f64};
    println!("{r:?}");
    println!("{}", r.area());
    println!("{r:?}");
}
```



```
use std::f32::consts::PI;
use Forma::*;
#[derive(Debug)]
enum Forma {
    Circulo {raio:f64},
    Retangulo {alt:f64, compr:f64}
}
impl Forma {
    fn escala(&mut self, e:f64){
        match self {
            Circulo {raio: r} => {*r = *r * e;}
            Retangulo {alt: a, compr: c} => {*a = *a * e; *c = *c * e;}
        }
    }
}
fn main (){
    let mut c = Retangulo {alt:10f32, compr:5f32};
    println!("{c:?}");
    c.escala(2f32);
    println!("{c:?}");
}
```

 &mut f64

[illegible]